

CMSE 801 - Day 20



Defining and using Python classes

November 11, 2024

General Course Announcements

- Homework 4 is due this **Friday, November 15, at 11:59pm**.
- I've graded Quiz 3 and will be uploading grade to D2L later today.
- Project Draft Presentations will be during class on **Wednesday, November 20** (Day 23).
 - More details to come!

Questions/comments from Day 20 PCA

- A number of questions about how to solve "Question 8" -- **we'll look at this briefly.**
- General awareness that getting used to the Python `class` syntax will take some time, including the purpose of things like `self` and `__init__()`. Additionally, understanding the process for defining attributes vs. methods was highlighted as an issue.
 - **We'll take a look at this before we get started.**
- There was a lot of new information to process in this PCA
 - Similar to when you first learned things like writing Python loops or functions, it will take time to digest this new information and learn the syntax. This is part of the process of expanding your Python toolbox!

Question 8: Write a bit of code that converts first 5 words to upper case and prints them out.

- first with lists and loops (**Hint:** split string to words and convert first 5 into upper case)
- try to do it using methods from `string` class (**Hint:** convert the whole string into upper case and split it into words and print the first 5 words)

```
In [1]: text = "In this assignment, you are learning about Object Oriented Programming"

# Converting the first 5 words to uppercase
# using loops and lists
wordsUpper = []
for word in text.split(" "):
    wordsUpper.append(word.upper())
print("Words:", wordsUpper)
```

```
Words: ['IN', 'THIS', 'ASSIGNMENT,', 'YOU', 'ARE']
```

```
In [2]: # Taking even more of an OOP approach
wordsUpper = text.upper().split(" "):5]
print("Words:", wordsUpper)
```

```
Words: ['IN', 'THIS', 'ASSIGNMENT,', 'YOU', 'ARE']
```

Let's practice some of these concepts with a simple class

Remember: A class is made of three main things: name, attributes, and methods

```
In [3]: class Student():
        '''
        This class is designed to include information about individual students
        Currently this class has the following attributes:

        name : this is the student's name
        gpa : this is the student's current gpa
        year : this is what year of college the student is in
        '''
        def __init__(self, name='', gpa=0.0, year=1):
            self.name = name
            self.gpa = gpa
            self.year = year
```

```
In [4]: theo = Student('Theodore', gpa=3.2)
```

```
In [5]: print(theo.name)
        print(theo.gpa)
        print(theo.year)
```

```
Theodore
3.2
1
```

Now we'll expand that class a bit with some methods.

```
In [6]: class Student():
        """
        This class is designed to include information about individual students.

        There are now also methods for interacting with the Student class.
        """
        def __init__(self, name='', gpa=0.0, year=1):
            self.name = name
            self.gpa = gpa
            self.year = year
            self.courses = []

        def get_name(self):
            print("My name is %s." %self.name)

        def enroll(self,courses):
            self.courses = courses

        def display_enrolled(self):
            if len(self.courses) == 0:
                print("I am not enrolled in any courses at the moment")
            else:
                print("I am enrolled in %s and %s." %(self.courses[0], self.courses[1]))

        def years_until_graduation(self):
            return 4 - self.year
```

And then we can test those new methods

```
In [7]: theo = Student('Theodore', gpa=3.2)
```

```
In [8]: theo.get_name()
```

My name is Theodore.

```
In [12]: theo.enroll(['CMSE 201', 'MTH 133'])  
theo.display_enrolled()
```

I am enrolled in CMSE 201 and MTH 133.

```
In [13]: theo.years_until_graduation()
```

```
Out[13]: 3
```

Note: Syntax is different for attributes vs. methods

```
In [14]: theo.name
```

```
Out[14]: 'Theodore'
```

```
In [15]: theo.name()
```

```
-----  
-----  
TypeError                                Traceback (most recent  
call last)  
Cell In[15], line 1  
----> 1 theo.name()  
  
TypeError: 'str' object is not callable
```

```
In [16]: theo.get_name
```

```
Out[16]: <bound method Student.get_name of <__main__.Student object at 0  
x110d1d220>>
```

```
In [17]: theo.get_name()
```

My name is Theodore.

How can we explore the bits and pieces built into the Student class?

```
In [18]: dir(theo) # Note: you can double-click on the output to collapse
```

Out[18]:

```
['__class__',
 '__delattr__',
 '__dict__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__le__',
 '__lt__',
 '__module__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 '__weakref__',
 'courses',
 'display_enrolled',
 'enroll',
 'get_name',
```

```
'gpa',  
'name',  
'year',  
'years_until_graduation']
```

```
In [ ]: # We can press tab after the "."  
theo.
```

```
In [19]: # We can pull up some of the documentation  
# (but this is only as good as the documentation we wrote)  
theo?
```

Goals for today

- Review code and use that uses built-in or custom-written Python classes (objects)
- Use objects in connection to other variable types in Python

Looking forward - Day 21 PCA: Inheritance and Composition with Python Classes/Objects

- You'll review what it means for one Python class to "inherit" the properties of another one.
- You'll explore how objects can be "composed" of other pre-existing objects
- You're experiment with defining and manipulating objects.

